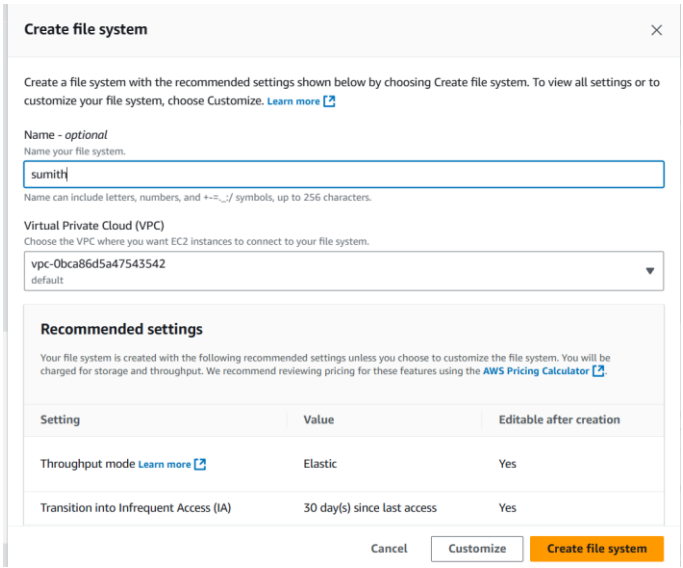
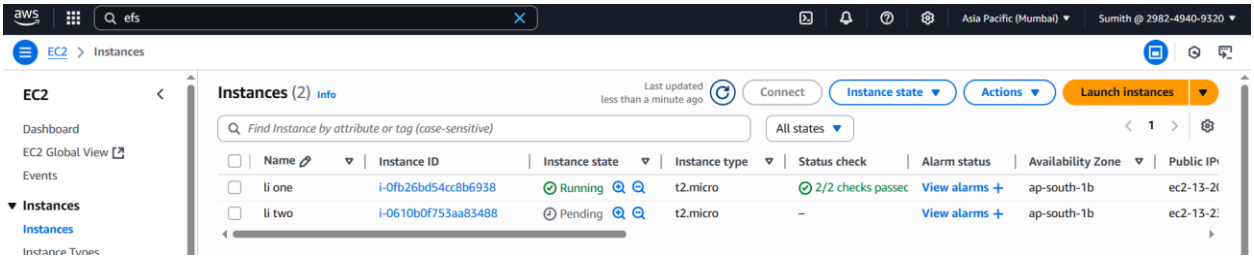
Mounting and Sharing EFS on Multiple Linux EC2 Instances

Easily share files between EC2s using AWS Elastic File System (EFS).

✅ Step 1: Create an EFS File System

1. Go to **AWS Console > EFS**.
2. Click **"Create file system"**.
3. Select **VPC** and **Availability Zones** where your EC2 instances exist.
4. Choose **mount targets** in the same subnets as your EC2 instances.
5. Use **default security group** (or custom, but allow NFS - port 2049).
6. Finish creation.





✅ Step 2: Configure Security Group (Very Important)

Update the security group attached to **both EC2 instances** and **EFS mount target**:

- Allow **inbound NFS (TCP 2049)** from the **EC2 security group**.
- Allow **outbound access** from EC2 to EFS (for DNS & mounting).

✅ **Step 3: Install NFS Utils on Both Linux EC2 Instances**

```
sudo yum update -y   # For Amazon Linux / RHEL
sudo yum install -y nfs-utils
```

or

```
sudo apt update      # For Ubuntu
sudo apt install -y nfs-common
```

✅ **Step 4: Mount EFS on First EC2 Instance**

1. Create a directory:

   ```
   sudo mkdir /mnt/efs
   ```

2. Mount EFS (replace fs-xxxx with your EFS ID):

   ```
   sudo mount -t nfs4 -o nfsvers=4.1 fs-xxxx.efs.<region>.amazonaws.com:/ /mnt/efs
   ```

3. Confirm:

   ```
   df -h
   ```

✅ **Step 5: Add Some Shared Files**

```
echo "Hello from EC2-A" | sudo tee /mnt/efs/hello.txt
```

✅ **Step 6: Mount EFS on Second EC2 Instance (Same Steps)**

1. Install NFS utils (if not done).
2. Create mount point:

   ```
   sudo mkdir /mnt/efs
   ```

3. Mount the same EFS:

   ```
   sudo mount -t nfs4 -o nfsvers=4.1 fs-xxxx.efs.<region>.amazonaws.com:/ /mnt/efs
   ```

✅ **Step 7: Verify File Sharing**

On EC2-B:

bash

CopyEdit

```
cat /mnt/efs/hello.txt
```

You should see: Hello from EC2-A

✅ Step 8: (Optional) Add to fstab for Auto-mount on Reboot

Add this line to /etc/fstab on both instances:

bash

CopyEdit

fs-xxxx.efs.<region>.amazonaws.com:/ /mnt/efs nfs4 defaults,_netdev 0 0

```
Get:50 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [4288 B]
Get:51 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:52 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [380 B]
Fetched 35.8 MB in 34s (1058 kB/s)
Reading package lists... Done
root@ip-172-31-2-79:~# hostnamectl set-hostname mc1
root@ip-172-31-2-79:~# reboot now

Broadcast message from root@ip-172-31-2-79 on pts/1 (Tue 2025-07-29 03:22:24 UTC):

The system will reboot now!

root@ip-172-31-2-79:~# client_loop: send disconnect: Connection reset

C:\Users\WELCOME\Downloads>ssh -i "ebs.pem" ubuntu@ec2-13-201-131-251.ap-south-1.compute.amazonaws.com
```

```
roc-fs-nfsd.mount is a disabled or a static unit, not starting it.
pc-gssd.service is a disabled or a static unit, not starting it.
pc-statd-notify.service is a disabled or a static unit, not starting it.
pc-statd.service is a disabled or a static unit, not starting it.
pc-svcgssd.service is a disabled or a static unit, not starting it.
rocessing triggers for man-db (2.12.0-4build2) ...
rocessing triggers for libc-bin (2.39-0ubuntu8.4) ...
canning processes...
canning linux images...

unning kernel seems to be up-to-date.

o services need to be restarted.

o containers need to be restarted.

o user sessions are running outdated binaries.

o VM guests are running outdated hypervisor (qemu) binaries on this host.
buntu@mc1:~$ df -h
ilesystem      Size  Used Avail Use% Mounted on
dev/root       6.8G  2.0G  4.8G  30% /
mpfs           479M     0  479M   0% /dev/shm
mpfs           192M  884K  191M   1% /run
mpfs           5.0M     0  5.0M   0% /run/lock
dev/xvda16     881M   86M  734M  11% /boot
dev/xvda15     105M  6.2M   99M   6% /boot/efi
mpfs            96M   12K   96M   1% /run/user/1000
buntu@mc1:~$
```

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@mc2:~$ sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2
ubuntu@mc2:~$ df -h
Filesystem                                          Size  Used Avail Use% Mounted on
/dev/root                                           6.8G  2.0G  4.8G  30% /
tmpfs                                               479M     0  479M   0% /dev/shm
tmpfs                                               192M  884K  191M   1% /run
tmpfs                                               5.0M     0  5.0M   0% /run/lock
/dev/xvda16                                         881M   86M  734M  11% /boot
/dev/xvda15                                         105M  6.2M   99M   6% /boot/efi
tmpfs                                                96M   12K   96M   1% /run/user/1000
fs-0da5f4893b1198cf8.efs.ap-south-1.amazonaws.com:/  8.0E     0  8.0E   0% /home/ubuntu/efs
ubuntu@mc2:~$ cd efs
ubuntu@mc2:~/efs$ ls
fold  hit.txt
ubuntu@mc2:~/efs$
```

```
ubuntu@mc1:~/efs$ ls -a
.  ..
ubuntu@mc1:~/efs$ sudo -i
root@mc1:~# cd efs
-bash: cd: efs: No such file or directory
root@mc1:~# exit
logout
ubuntu@mc1:~/efs$ cd ..
ubuntu@mc1:~$ sudo chown -R ubuntu:ubuntu efs/
ubuntu@mc1:~$ ls -la
total 40
drwxr-x--- 6 ubuntu ubuntu 4096 Jul 29 03:44 .
drwxr-xr-x 3 root   root   4096 Jul 29 03:17 ..
-rw------- 1 ubuntu ubuntu  449 Jul 29 03:45 .bash_history
-rw-r--r-- 1 ubuntu ubuntu  220 Mar 31  2024 .bash_logout
-rw-r--r-- 1 ubuntu ubuntu 3771 Mar 31  2024 .bashrc
drwx------ 2 ubuntu ubuntu 4096 Jul 29 03:20 .cache
drwxrwxr-x 3 ubuntu ubuntu 4096 Jul 29 03:44 .local
-rw-r--r-- 1 ubuntu ubuntu  807 Mar 31  2024 .profile
drwx------ 2 ubuntu ubuntu 4096 Jul 29 03:17 .ssh
-rw-r--r-- 1 ubuntu ubuntu    0 Jul 29 03:20 .sudo_as_admin_successful
drwxr-xr-x 2 ubuntu ubuntu 6144 Jul 29 03:13 efs
ubuntu@mc1:~$ cd efs/
ubuntu@mc1:~/efs$ mkdir fold
ubuntu@mc1:~/efs$ touch hit.txt
ubuntu@mc1:~/efs$ nano hit.txt
ubuntu@mc1:~/efs$ ubuntu@mc1:~/efs$ cat hit.txt
deek helped me

ubuntu@mc1:~/efs$
```

◈ Mounting Amazon EFS on Windows Server (NFSv4.1)

✅ Objective:

Mount EFS to Windows Server using NFS v4.1 and enable file sharing between Windows instances.

✅ Prerequisites

- ✅ Two Windows Server 2022/2025 EC2 instances.
- ✅ Amazon EFS created and available.
- ✅ Same **VPC & subnet** as EC2.
- ✅ Security groups allow **NFS port 2049**.
- ✅ Both EC2s have **NFS client support enabled**.

⚙ Step-by-Step Process

Step 1: Enable NFS Client on Windows

On **each EC2 Windows instance**, run:

powershell

CopyEdit

```
dism /online /Enable-Feature /FeatureName:NFS-Client
```

Reboot if prompted.

Step 2: Create a Mount Point Directory

Open **PowerShell** or **CMD**:

cmd

CopyEdit

```
mkdir Z:\EFS
```

(Or use C:\efs or any other path)

Step 3: Mount EFS via NFSv4.1

Use your EFS ID and region:

cmd

CopyEdit

```
mount -o mtype=hard rsize=1048576,wsize=1048576,timeout=600,retrans=2 fs-xxxx.efs.<region>.amazonaws.com:/ Z:
```

Notes:

- Z: is the new drive letter.
- fs-xxxx.efs.<region>.amazonaws.com:/ is your EFS DNS name.

Step 4: Validate the Mount

- Open **File Explorer**.
- You should see *Z:* mounted.
- Create a file and verify it exists.

Step 5: Mount on Second Windows EC2

Repeat Steps 1–4 on the second Windows Server.

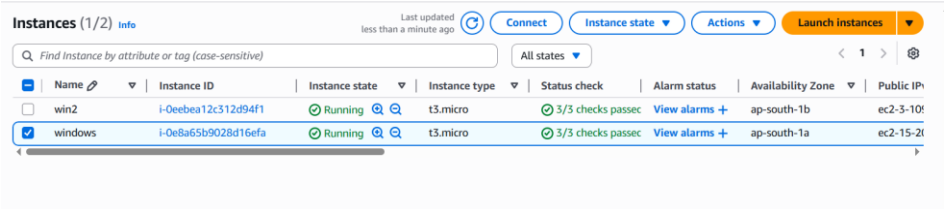
Try to access the file created from the first EC2. If visible – ✅ it's working.

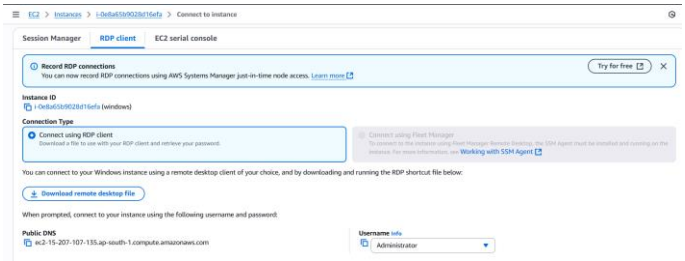
✅ (Optional) Auto-Mount on Reboot

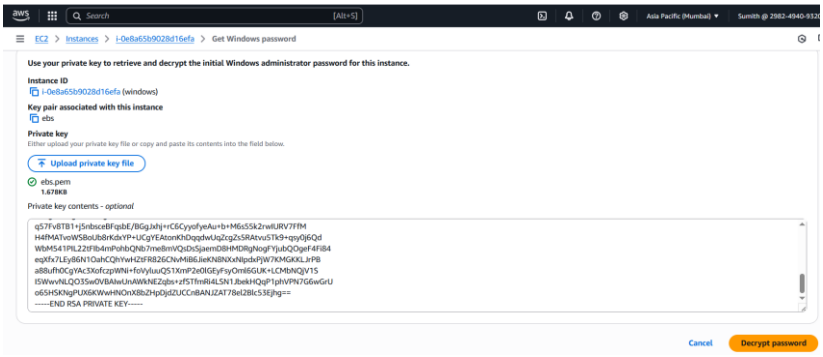
Use **Task Scheduler** or a **Startup script** to re-run the mount command at boot, since Windows does not persist NFS mounts across reboot.

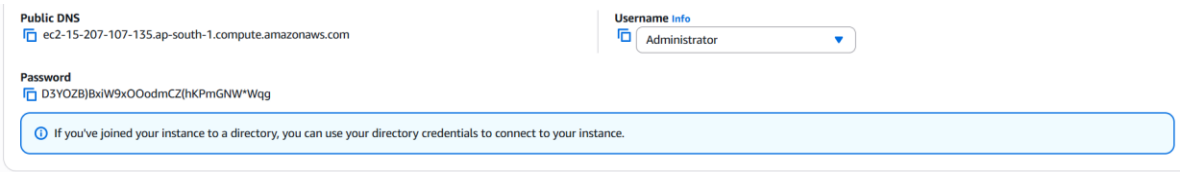
🔐 Security Group Reminder

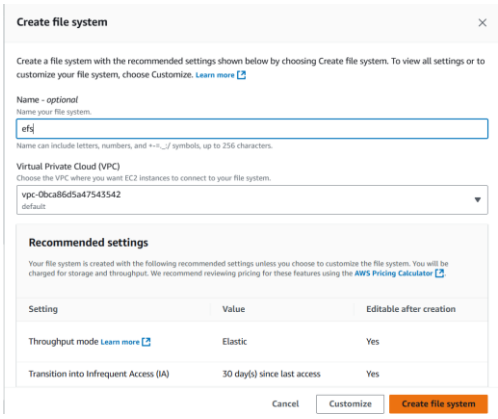
- Inbound Rule: TCP **2049** from **Windows EC2 SG**.
- Outbound: Allow all (or specific EFS endpoints

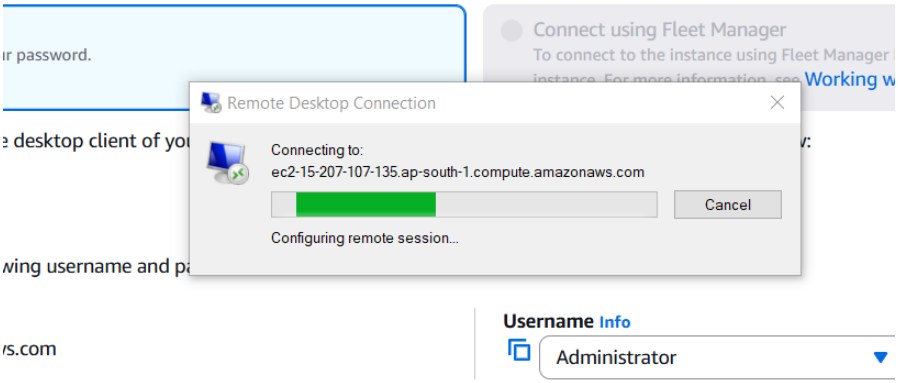


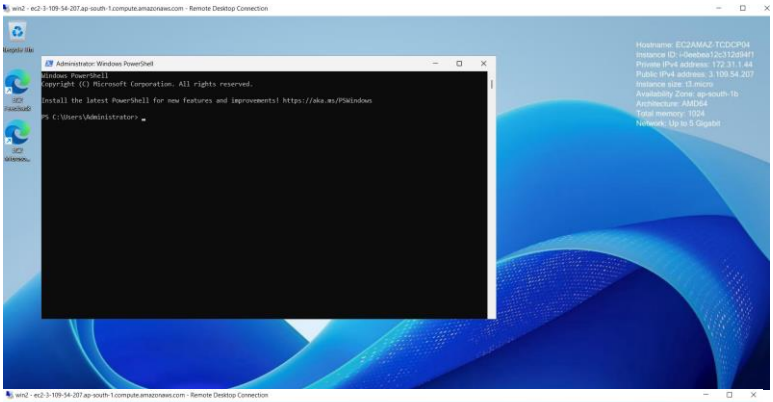


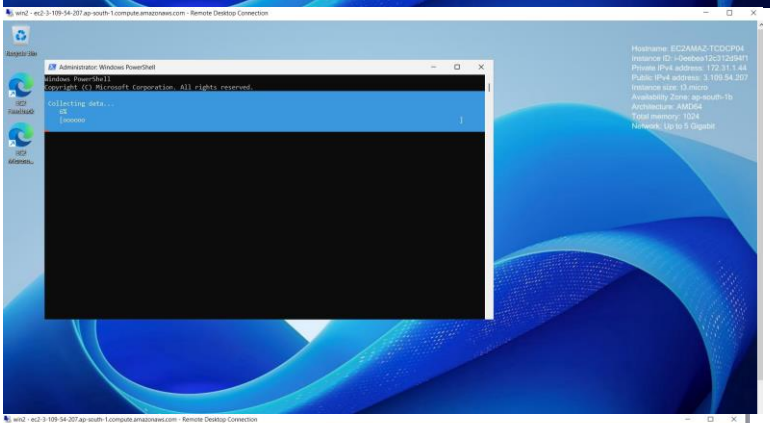

EC2 > Instances > i-0e8a65b9028d16efa > Get Windows password
Use your private key to retrieve and decrypt the initial Windows administrator password for this instance.
Instance ID
i-0e8a65b9028d16efa (windows)
Key pair associated with this instance
ebs
Private key
Either upload your private key file or copy and paste its contents into the field below.
Upload private key file
ebs.pem
1.678KB
Private key contents - optional
-----END RSA PRIVATE KEY-----
Cancel
Decrypt password

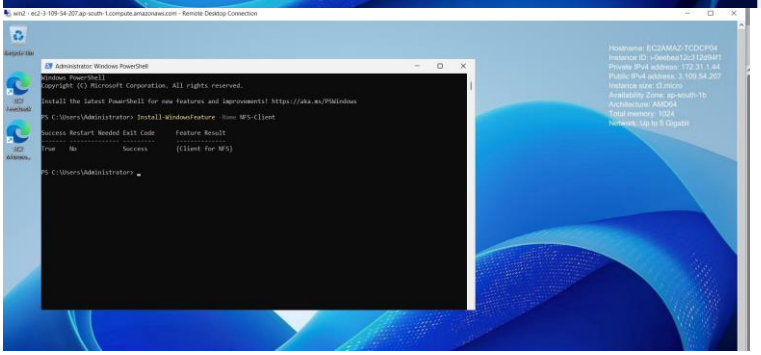

Public DNS
ec2-15-207-107-135.ap-south-1.compute.amazonaws.com
Username Info
Administrator
Password
D3YOZB)BxiW9xOOodmCZ(hKPmGNW*Wqg
If you've joined your instance to a directory, you can use your directory credentials to connect to your instance.


Create file system
Create a file system with the recommended settings shown below by choosing Create file system. To view all settings or to customize your file system, choose Customize. Learn more
Name - optional
Name your file system.
efs
Name can include letters, numbers, and +-=._:/ symbols, up to 256 characters.
Virtual Private Cloud (VPC)
Choose the VPC where you want EC2 instances to connect to your file system.
vpc-0bca86d5a47543542
default
Recommended settings
Your file system is created with the following recommended settings unless you choose to customize the file system. You will be charged for storage and throughput. We recommend reviewing pricing for these features using the AWS Pricing Calculator.
Setting
Value
Editable after creation
Throughput mode Learn more
Elastic
Yes
Transition into Infrequent Access (IA)
30 day(s) since last access
Yes
Cancel
Customize
Create file system


r password.
Connect using Fleet Manager
To connect to the instance using Fleet Manager
Working w
Remote Desktop Connection
Connecting to:
ec2-15-207-107-135.ap-south-1.compute.amazonaws.com
Cancel
Configuring remote session...
e desktop client of you
wing username and pa
s.com
Username Info
Administrator







NOTE:

Got few error in the connectivity of the nfs over the windows server 2025. Need to re-verify the process & get back for detailed documentation.

Optional:

🔧 **1. Troubleshooting NFS Permissions**

🔹 **Task: Simulate and resolve UID/GID-based permission errors.**

✅ **Step-by-step (Linux EC2)**

A. Intentionally misconfigure permissions

1. Mount EFS:

bash

CopyEdit

sudo mount -t nfs4 -o nfsvers=4.1 fs-xxxx.efs.region.amazonaws.com:/ /mnt/efs

2. Create a file as root:

bash

CopyEdit

sudo touch /mnt/efs/root-owned.txt

sudo chown root:root /mnt/efs/root-owned.txt

3. Try accessing as **non-root** user:

bash

CopyEdit

sudo -u ec2-user cat /mnt/efs/root-owned.txt

→ Permission denied

✅ **B. Fix Using EFS Access Point (Recommended)**

1. Go to **EFS > Access Points > Create Access Point**.
2. Set:
   - POSIX user: UID=1000, GID=1000
   - Root directory: /shared
   - Permissions: 0777
3. Mount using Access Point:

bash

CopyEdit

sudo mount -t nfs4 -o nfsvers=4.1,tls fs-xxxx.efs.region.amazonaws.com:/access-point-id /mnt/efs

✅ **Learning**: NFS does not translate Linux UIDs/GIDs. If EC2 user doesn't match EFS file ownership, permission issues occur. Access Points abstract this away.

⚙️ 2. Automating EFS Mounts on Boot

✅ Linux EC2 via User Data

bash

CopyEdit

```
#!/bin/bash
yum install -y amazon-efs-utils
mkdir /mnt/efs
echo "fs-xxxx.efs.region.amazonaws.com:/ /mnt/efs efs defaults,_netdev 0 0" >> /etc/fstab
mount -a
```

✅ Launch EC2 with this as **User Data script**.

✅ Windows EC2 via PowerShell Script

powershell

CopyEdit

```
$efsDns = "fs-xxxx.efs.region.amazonaws.com"
$mountPoint = "Z:"
if (-not (Test-Path $mountPoint)) { New-Item -ItemType Directory -Force -Path $mountPoint }
mount -o mtype=hard $efsDns:/ $mountPoint
```

✅ Use **Task Scheduler** or EC2 User Data (PowerShell) to run on boot.

💰 3. EFS Cost Optimization

🔹 Compare Storage Classes:

Class	Use Case	Cost	Latency
Standard	Frequent access	Higher	Low
Infrequent Access (IA)	Rare access	92% cheaper	Slightly higher

---

✅ **Create Lifecycle Policy**

1. Go to **EFS > File Systems > Lifecycle Policy**.
2. Set:
   - Transition to IA after **30 days of no access**

✅ **CLI Option**

bash

CopyEdit

```
aws efs put-lifecycle-configuration \
 --file-system-id fs-xxxx \
 --lifecycle-policies '[{"TransitionToIA":"AFTER_30_DAYS"}]'
```

✅ **Learning**: Lifecycle policies automate cost-saving without changing app logic.

📓 **Summary Poster Headings**

- **Understand EFS Permissions with NFS (Linux & Windows)**
- ⚙️ **Auto-Mount EFS: Boot-time Setup for EC2**
- 💸 **Cut Costs: Use Lifecycle Policies & IA Class in EFS**

DOUBT CLARIFICATION CONCEPTIVE Q/A:

□ Section: NFS on Linux

◆ Q1: How does NFS work on Linux in the context of AWS EFS?

Answer:

NFS (typically version 4.1 in AWS EFS) allows Linux instances to mount EFS file systems over the network as if they were local directories. EFS uses NFSv4.1 protocol and supports POSIX file semantics, meaning it respects Linux file permissions (UID/GID). You mount EFS using the mount command or /etc/fstab and ensure the EC2 has nfs-utils installed and access to port 2049.

◆ Q2: Why do Linux users face permission denied errors when accessing EFS?

Answer:

EFS uses POSIX-compliant UID and GID for permissions. If the EC2 Linux user's UID/GID doesn't match the file owner UID/GID on EFS, access will be denied—even if usernames match. This is because EFS doesn't map usernames but uses numeric IDs. This issue is resolved using **EFS Access Points**, which allow setting a default UID/GID when mounting.

◆ Q3: What are the key options when mounting EFS on Linux?

Answer:

- nfsvers=4.1: Use NFS version 4.1
- tls: Enables encrypted transport
- _netdev: Tells system not to mount until network is up (for fstab)
- rsize, wsize: Read/write buffer sizes for tuning
- Example:

bash

CopyEdit

```
sudo mount -t nfs4 -o nfsvers=4.1,tls,rsize=1048576,wsize=1048576 fs-xxx:/ /mnt/efs
```

□ Section: NFS on Windows Server

◆ Q4: Can Windows Server mount EFS using NFS? If so, how?

Answer:

Yes. Windows Server (2022/2025) supports mounting EFS using **NFSv4.1**, but requires enabling the **NFS Client** feature first. After that, the mount command is used with EFS's DNS name. EFS will appear as a drive (e.g., Z:\) in File Explorer. However, POSIX permissions can still interfere with read/write access.

◆ Q5: What are the challenges when using EFS with Windows Server?

Answer:

- **Permission Mismatch**: EFS uses POSIX (UID/GID), while Windows uses SIDs, causing access errors.
- **Limited NFS tooling**: Windows lacks the same level of NFS troubleshooting/logging tools as Linux.
- **No persistent mount**: Windows does not auto-mount NFS shares after reboot without scripting.

Workarounds include using **EFS Access Points** or ensuring that the root directory in EFS has permissions 0777.

◆ Q6: How do you persist NFS mount across reboots in Windows?

Answer:

Use a **PowerShell script** triggered on boot using **Task Scheduler** or **EC2 User Data**. Example:

powershell

CopyEdit

mount -o mtype=hard fs-xxxx.efs.region.amazonaws.com:/ Z:

Alternatives include New-PSDrive, but it’s more suitable for SMB, not NFS.

□ Section: Cross-Platform & Best Practices

◆ Q7: What is the purpose of using EFS Access Points in mixed environments (Linux + Windows)?

Answer:

EFS Access Points simplify permission management by allowing you to:

- Set a default UID/GID and root directory.
- Abstract POSIX permission issues for apps or users that do not control UID/GID (e.g., Windows).
- Allow secure, isolated, and user-specific access even in shared environments.

◆ Q8: How can you secure EFS in an NFS setup?

Answer:

- Use **NFS over TLS** for encrypted traffic.
- Restrict **security group** to allow port 2049 only from trusted instances.

- Use **EFS access policies** and **IAM** to define access.
- Enforce permissions at **Access Point level**, not just file system.

◆ Q9: What's the difference between EFS and NFS?

Answer:

EFS is **AWS's managed file system** that **uses NFSv4.1 protocol**. NFS is just a **protocol**—EFS handles scalability, availability, and encryption. EFS automates what traditional NFS servers required manually (e.g., failover, backups, access control, etc.).

◆ Q10: When should you avoid using EFS with NFS?

Answer:

- When **latency-sensitive** workloads (e.g., databases) require faster IOPS than EFS offers.
- When **Windows-native apps** require NTFS permissions or SMB protocol.
- If using containers without POSIX UID/GID awareness and no EFS Access Points.